Technologie du commerce électronique

INF27523 (MS)

UNIVERSITÉ DU QUÉBEC À RIMOUSKI

DÉPARTEMENT DE MATHÉMATIQUES, D'INFORMATIQUE ET DE GÉNIE

RAPPORT

Travail Pratique 01

ÉQUIPE :
Amina Missoum
Émile Santerre
Léa Esso-Solam Bikassa

Chargée de cours :
Oumeima Nuigues

Table des matières

1 Introduction 3
2 Description du projet 3
3 Technologies utilisées 4
4 Mise en œuvre et fonctionnalités 5
5 Défis rencontrés 7
6 Conclusion 8

1 Introduction

Dans le cadre du cours de commerce électronique, ce travail pratique avait pour objectif de concevoir et d'implémenter une application web de boutique en ligne, en mettant en pratique les notions vues en classe autour des architectures MVC, de la gestion de sessions et de l'intégration d'une passerelle de paiement. Nous avons développé un site permettant à un client de naviguer parmi les restaurants, de consulter leur menu, de constituer un panier, puis de finaliser sa commande à l'aide d'un paiement sécurisé avec Stripe.

Au-delà de la prise de commande, le projet nous a permis d'aborder plusieurs aspects clés d'une application de commerce électronique : gestion de comptes utilisateurs (client et vendeur), association des restaurants à un vendeur, validation métier (un panier ne peut contenir que des plats d'un seul restaurant), gestion du panier en session, historique des commandes et interface dédiée aux restaurateurs pour consulter leurs ventes. Ce rapport présente la description globale du projet, les technologies utilisées, la mise en œuvre des principales fonctionnalités, les défis rencontrés ainsi que les apprentissages que nous en retirons.

2 Description du projet

Le projet consiste en la réalisation d'un site web de commande de repas nommé « ÉléganceShop », inspiré d'une plateforme de type agrégateur de restaurants, mais centré sur un ensemble restreint de restaurants. L'application vise à offrir à un client une expérience fluide allant de la découverte du menu jusqu'au paiement en ligne, tout en fournissant aux vendeurs un accès simple aux commandes reçues.

Plus concrètement, nous avons voulu :

- Proposer une interface claire pour parcourir les restaurants et leurs plats.
- Permettre la création de comptes Client et Vendeur, avec un rôle bien défini.
- Une liste des jeux de données avec recherche et filtres dynamiques.
- Gérer un panier persistant, lié à la session, avec calcul automatique du total.
- Intégrer un paiement réel via Stripe pour simuler un scénario de production.
- Enregistrer les commandes et permettre au restaurant associé d'en consulter le détail.

2.2 Contexte fonctionnel

D’un point de vue opérationnel, l'application se divise en deux parcours utilisateurs. Pour le client, le scénario commence par la création d'un compte ou une authentification sécurisée. Une fois connecté, l'utilisateur navigue parmi les restaurants disponibles, consulte les cartes détaillées et compose son panier. Le processus termine lors de la phase de validation où le client accède à une interface de paiement pour confirmer sa transaction avant de pouvoir suivre l'état de sa commande dans son historique personnel.

À l'opposé, le vendeur bénéficie d'une interface d'administration. Lors de son inscription, celui-ci est obligatoirement associé à un établissement existant, ce qui lui octroie le droit de consulter la liste des commandes reçues. Cette interface permet au restaurateur d'accéder aux détails de chaque vente, tels que la liste des plats commandés, les quantités exactes, les montants perçus ainsi que les informations de contact du client.

2.3 Fonctionnalités principales

La mise en œuvre technique de l'application repose sur plusieurs piliers fonctionnels interconnectés. La gestion des comptes constitue la première brique, utilisant une authentification par cookies et des rôles bien définis (Client ou Vendeur) pour restreindre l'accès aux fonctionnalités spécifiques. L'exploration des restaurants a été optimisée par la création de pages détaillées affichant les menus, les descriptions de chaque plat.

Le cœur de l'application réside cependant dans la logique du panier d'achat. Ce dernier n'est pas seulement un espace de stockage temporaire, il intègre une règle stricte qui limite le contenu à un seul restaurant par commande pour garantir la cohérence logistique. Cette contrainte est gérée par un système de détection de conflit qui alerte l'utilisateur en cas d'ajout incohérent. Enfin, le processus de paiement est sécurisé par Stripe Elements, assurant que les données bancaires ne transitent jamais en clair sur notre serveur. Une fois le paiement validé, le système génère automatiquement une commande et une facture, mettant ainsi à jour l'historique du client et le tableau de bord des ventes du vendeur.

3 Technologies utilisées

3.1 Environnement de développement

Pour ce projet nous nous sommes appuyés sur un écosystème d'outils professionnels. Le développement a été principalement orchestré sous Visual Studio, permettant une gestion fluide du code C#, des vues Razor et des feuilles de style CSS. Pour assurer la durabilité du travail et faciliter les phases d'intégration, nous avons utilisé Git avec GitHub comme système de gestion de versions. Cette approche nous a permis de travailler sur des branches distinctes, notamment pour isoler les tests de l'API Stripe avant de fusionner les fonctionnalités validées dans la branche principale. L'aspect technique repose sur le framework ASP.NET Core en MVC,

qui structure l'application selon un modèle de séparation des responsabilités, tandis qu'Entity Framework Core assure la liaison entre le code et la base de données relationnelle.

3.2 Langages et frameworks

Notre application est propulsée par le langage C# via le framework ASP.NET Core, qui gère l'ensemble de la logique du serveur, des contrôleurs spécialisés aux mécanismes de routage. Pour la partie interface, nous avons privilégié Razor, facilitant la génération dynamique de contenu HTML à partir des données du modèle, comme l'affichage des paniers ou des historiques de commandes.

La persistance des données est opérée par Entity Framework Core, agissant comme une couche d'abstraction pour manipuler les entités telles que les utilisateurs, les menus et les factures sans injection SQL directe. Le volet transactionnel est quant à lui sécurisé par Stripe .NET et Stripe.js, offrant une gestion des intentions de paiement. Enfin, l'aspect visuel et l'ergonomie ont été standardisés grâce à Bootstrap 5, complété par Bootstrap Icons pour une navigation intuitive, le tout soutenu par une charte graphique personnalisée en HTML5 et CSS3.

3.3 Structure et services

Notre architecture interne a été pensée pour favoriser la modularité et la réutilisation du code. Nous avons notamment mis en place un service dédié, le CartService, dont la responsabilité unique est d'orchestrer la logique du panier d'achat de sa création initiale en session jusqu'à la gestion des incrémentations de quantités et de la suppression d'items.

Ce service interagit avec un modèle de données structuré où chaque entité possède des relations claires. Un restaurant rassemble plusieurs plats et se lie à un vendeur unique, tandis qu'un panier regroupe des éléments référençant des produits spécifiques. Cette organisation rigoureuse entre les entités User, Restaurant, Plat et Order permet de maintenir une intégrité des données, particulièrement lors de la transformation finale d'un panier en commande après validation du paiement.

4 Mise en œuvre et fonctionnalités

4.1 Organisation des contrôleurs et des vues

L’architecture de l’application a été segmentée en plusieurs pôles de responsabilité afin de garantir une maintenance aisée et une séparation claire des préoccupations. Le cœur de l’expérience utilisateur est piloté par le HomeController et le RestaurantsController, qui gèrent respectivement la vitrine de la plateforme et l'affichage des menus. La gestion sensible des

accès et des comptes est isolée dans l'AccountController, tandis que le cycle de vie de la commande est réparti entre le CartController pour la phase de sélection, le CheckoutController pour la transaction financière, et les contrôleurs OrdersController et SalesController pour le suivi post-achat. Chaque contrôleur communique avec des vues Razor spécifiques, structurées pour offrir une interface cohérente. L'utilisation d'un fichier de style global assure que les composants visuels, tels que les cartes de restaurants ou les résumés de commandes, conservent une unité esthétique sur l'ensemble du site.

4.2 Authentification, rôles et profil

La sécurité et la personnalisation de l'expérience reposent sur un système d'authentification basé sur les identités stockées en base de données. Lors de la phase d'inscription, l'utilisateur se voit assigner un rôle spécifique (Client ou Vendeur) qui détermine ses droits d'accès au sur de la plateforme. Le processus de connexion utilise des cookies d'authentification pour maintenir la session active, permettent de vérifier instantanément le rôle de l'utilisateur lors de chaque requête. Cette structure permet, par exemple, de restreindre l'accès au formulaire de paiement aux seuls clients, ou de réserver les suivis de vente au restaurateur approprié. Une page de profil dédiée permet aussi à chaque utilisateur de gérer ses informations personnelles, garantissant ainsi la conformité aux standards de gestion de comptes modernes.

4.3 Gestion du panier et règle métier

Le panier d’achat constitue l’élément le plus complexe de la logique métier. Pour éviter toute confusion lors de la préparation des commandes, nous avons instauré une contrainte technique interdisant le mélange de plats provenant de différents établissements. Cette règle est pilotée par le CartService qui, lors de chaque ajout, compare l'identifiant du restaurant du nouveau plat avec celui des articles déjà présents. Si une incohérence est détectée, le système lève une exception qui est interceptée par le contrôleur pour déclencher une interface modale. Cette fenêtre contextuelle offre au client une alternative claire, abandonner l'ajout pour préserver ses choix actuels, ou réinitialiser complètement son panier pour entamer une nouvelle commande auprès du second restaurant. Cette approche garantit que chaque commande générée est dirigée vers un seul et unique restaurant.

4.4 Intégration de Stripe

Le déploiement de la solution de paiement s’est construit autour d'une communication bidirectionnelle entre notre serveur et les API de Stripe. La procédure débute par la création d'une intention de paiement (PaymentIntent) dès que le client accède à la page de validation, calculant le montant exact en cents pour éviter les erreurs d'arrondi. Côté client, nous utilisons Stripe Elements pour injecter un champ de saisie de carte bancaire sécurisé, ce qui permet de traiter les informations sensibles sans qu'elles ne transitent par nos propres serveurs, assurant ainsi une conformité PCI. Une fois que Stripe confirme la validité de la transaction, un identifiant unique est renvoyé à notre contrôleur. Ce dernier effectue une vérification finale de sécurité côté serveur avant de confirmer la création de la commande et de vider le panier, transformant ainsi une intention d'achat en une transaction finalisée et enregistrée.

5 Défis rencontrés

5.1 Gestion du panier et résolution des conflits de restaurants

L'un des défis techniques les plus important a été de maintenir la cohérence du panier face aux comportements imprévus des utilisateurs. Initialement, le système permettait l'ajout de plats issus de différents établissements, ce qui créait une ambiguïté majeure lors de la répartition des paiements et de la notification aux restaurateurs. Pour remédier à cela, nous avons conçu une logique d'exception personnalisée au sein du CartService. La difficulté ne résidait pas seulement dans l'interdiction de l'ajout, mais dans la gestion de l'expérience utilisateur qui en découle. Il a fallu orchestrer une communication entre le service, le contrôleur et une vue partielle asynchrone pour que la modale de conflit puisse offrir une option de réinitialisation automatique du panier. Ce mécanisme a nécessité une réflexion approfondie sur la gestion des états de session et sur la persistance des données temporaires.

5.2 Intégration de la passerelle Stripe et synchronisation

L'intégration d'une solution de paiement tierce comme Stripe a introduit une complexité supplémentaire liée à la synchronisation entre le front-end et le back-end. L'un des obstacles majeurs a été de garantir que le cycle de vie du PaymentIntent corresponde exactement aux actions de l'utilisateur. Nous avons dû faire face à des situations où le script Stripe.js ne se chargeait pas correctement ou dont le rendu visuel était altéré par des contraintes CSS, rendant le champ de saisie invisible ou inutilisable. De plus, la sécurisation du processus a exigé une validation rigoureuse côté serveur, il était essentiel de confirmer le statut du paiement directement via l'API Stripe avant de valider la commande en base de données, afin d'éviter la création de commandes "fantômes" en cas d'interruption de la connexion ou de fraude potentielle.

5.3 Sécurisation des accès et gestion des rôles

La mise en œuvre d'un système à deux types d'utilisateurs (Client et Vendeur) a apporté des enjeux de sécurité importants. Le défi consistait à restreindre l'accès aux URLs sensibles pour éviter qu'un client ne puisse visualiser les ventes d'un restaurant. Nous avons rencontré des difficultés initiales avec la mise à jour des "Claims" d'identité lors de la transition entre les comptes, ce qui provoquait parfois des erreurs de redirection. La résolution de ce problème a nécessité une configuration minutieuse du middleware d'authentification et l'application systématique de l'attribut [Authorize(Roles = "...")] sur les action du contrôleur, garantissant ainsi une forme d'étanchéité entre les différentes interfaces de l'application.

6 Conclusion

Ce travail pratique nous a permis de concevoir une application de commerce électronique complète, ancrée dans un scénario réel de restauration moderne. En articulant le projet autour d'ASP.NET Core en MVC et d'Entity Framework Core, nous avons pu transformer des concepts théoriques tels que la gestion d'états via les sessions en une solution fonctionnelle. L'intégration d'une passerelle de paiement réelle avec Stripe a constitué une étape déterminante, nous confrontant aux exigences de sécurité et de synchronisation propres aux transactions financières en ligne.

Au-delà des simples fonctionnalités de navigation et de commande, le projet nous a imposé une rigueur de structuration logicielle, notamment à travers le développement du CartService et la gestion fine des rôles utilisateurs. Les obstacles rencontrés, qu'il s'agisse de la logique de conflit entre restaurants ou des défis d'authentification, ont été des opportunités d'approfondir notre compréhension des mécanismes du Web. En conclusion, ce projet représente une immersion concrète dans les enjeux techniques et commerciaux du commerce électronique, nous offrant une base solide pour le développement de futures applications web professionnelles.